# 05 · 记忆与上下文管理

技术面试准备材料 · AI 应用 / Agent 开发

> 核心问题：什么该记住、记在哪里、什么时候取出来、什么时候该忘掉。

## 1. 为什么 Agent 需要记忆？记忆应该分几层？

因为模型本身是无状态的，每次调用只能看到当前上下文。没有记忆，`Agent` 就无法延续任务、复用经验，也无法理解「上次说到哪」。

我一般分四层：工作记忆、会话记忆、长期记忆、经验记忆。

工作记忆是当前这一步需要的信息，比如当前目标、已完成的步骤、最近的工具结果；会话记忆是这一轮对话的历史和状态；长期记忆是跨会话的用户偏好、项目事实、业务规则；经验记忆是成功/失败案例和复盘结论，用于以后复用。

分层的意义是成本和生命周期不同：工作记忆在上下文里，会话记忆可以落库，长期记忆要检索，经验记忆要结构化沉淀。不能把所有东西都塞进上下文，那样既贵又容易丢重点。

记忆系统的核心不是「存得多」，而是「在对的时候取出对的那一条」。

## 2. 短期记忆（会话上下文）怎么管理？

我会先区分「必须精确保留」和「可以压缩」的内容。

必须保留的：当前目标、用户明确约束、已确认的事实、未完成的待办、关键决策。这些用结构化状态保存，不依赖模型从历史里回忆。

可以压缩的：寒暄、重复确认、冗长的工具输出、早期的探索过程。压缩方式有滑动窗口、分段摘要、任务状态压缩。

压缩的触发点要明确：按 `Token` 阈值、按轮次、按任务阶段，而不是等报错。压缩后要保留来源和版本，避免信息丢了找不回来。

我还会做「工具结果裁剪」：只保留结论和证据，原始大结果放外部存储，需要时再取。

短期记忆的目标是「让模型看到当前任务最相关的最小集合」，不是「尽量多保留」。

## 3. 长期记忆怎么存？向量库、结构化存储还是知识图谱？

看记忆的类型。

用户偏好、配置、事实类信息适合结构化存储，字段明确、可查询、可更新；非结构化的对话摘要、文档片段适合向量检索；实体关系复杂、需要多跳推理的场景才考虑知识图谱。

实际系统通常是组合：结构化字段做精确过滤，向量做语义召回，图谱做关系推理。不要为了「高级」一上来就上图谱，维

护成本很高。

每条记忆至少要有：内容、来源、时间、置信度、归属（用户/租户/项目）、有效期。没有这些元数据，后面很难判断该不该信、该不该删。

存储之外还要有索引和生命周期策略：什么时候写、什么时候合并、什么时候过期。长期记忆最大的问题不是存不下，而是越存越脏。

## 4. 记忆应该在什么时候写入？

我会分显式和隐式两种。

显式写入：用户明确说「记住这个」，或者系统在关键节点主动保存，比如任务完成、偏好确认、重要决策。这类记忆置信度高，可以直接用。

隐式写入：从对话里推断偏好或事实。这类要谨慎，最好先记录为候选，经过多次确认或人工审核后再转正，避免把模型的猜测固化成事实。

写入前要做去重和冲突检查：相同内容合并，冲突内容保留版本和来源，不要直接覆盖。还要判断「这条信息是否值得长期保存」，临时任务状态不应该进长期记忆。

我的原则是：宁可少写、写准，也不要什么都存。记忆污染比没有记忆更难修。

## 5. 记忆检索怎么做？怎么决定取哪些？

我会综合四个信号：相关性、时效性、重要性、归属。

相关性用向量或关键词召回；时效性给新记忆更高权重，旧记忆降权或过滤；重要性由来源和类型决定，比如用户明确确认的偏好高于模型推断；归属做硬过滤，不能跨用户、跨租户取。

检索结果还要做预算控制：只取 `Top-K`，并且限制总 `Token`。取太多会挤占当前任务上下文，取太少又可能漏关键信息。

我通常会把「当前任务状态」和「检索到的记忆」分开标注，让模型知道哪些是当前事实、哪些是历史参考。冲突时以当前明确信息为准。

记忆检索不是一次性的，可以在任务过程中按需要多次取，而不是开头全量注入。

## 6. 记忆冲突、过期、错误怎么办？

首先，记忆要有版本和来源。冲突时不是简单覆盖，而是保留「谁在什么时候说了什么」，让系统或用户判断。

其次，要有置信度和有效期。用户明确确认的偏好置信度高；模型推断的低；临时信息带 `TTL`。过期后降权或删除，而不是继续当事实用。

第三，要能纠正。用户说「不对，现在已经改了」时，要能定位到旧记忆并标记失效，同时记录纠正原因。

第四，要能追溯。每条记忆能回答「从哪来、为什么信、影响了哪些输出」。出问题时可以回滚。

记忆系统的难点不是写入，而是治理。没有治理机制，记忆越多，错误越难发现。

## 7. 多用户、多租户下记忆怎么隔离?

隔离必须在存储和检索两层都做。

存储上，每条记忆带租户 ID 和用户 ID，物理上可以分库分表，逻辑上所有查询强制带过滤条件。不能依赖应用层「记得加 where」，要在数据访问层统一注入。

检索上，向量库的 metadata filter 必须包含租户和权限范围；缓存 key 也要带租户，避免跨租户命中。

对于共享知识和个人记忆要分开：共享知识走知识库，个人记忆走用户空间，权限不同、生命周期不同。

还要做越权测试和审计：验证 A 用户无法通过任何查询路径拿到 B 用户记忆；记录谁在什么时候访问了哪条记忆。

隐私要求高的场景，还要支持删除权：用户要求删除时，原文、向量、缓存、备份都要处理。

## 8. 上下文压缩为什么不可逆？怎么降低风险?

压缩是把多条原始信息变成更短的表示，这个过程一定会丢信息。摘要可能丢掉细节、限定条件、否定语气，甚至把「不确定」压成「确定」。一旦丢了，后面无法从摘要恢复。

降低风险的做法有四个。

第一，原始数据不删，只把压缩结果放进上下文；需要时可以回查原文。

第二，任务状态结构化保存，不依赖摘要。目标、约束、待办、已确认事实单独存字段。

第三，压缩策略可配置、可评测。不同任务用不同策略，关键任务少压缩或不压缩。

第四，压缩结果带来源和置信度，让模型知道这是摘要而不是原文。

我的原则是：压缩用于「节省上下文」，不用于「替代事实存储」。关键事实必须有独立、可追溯的存储。

## 9. 记忆和 RAG 有什么区别?

RAG 主要解决「外部知识」问题：文档、知识库、业务资料，通常是共享的、相对静态的，需要引用和权限控制。

记忆主要解决「这个用户/这个任务的历史」问题：偏好、已确认事实、之前的决策、经验教训，通常是个性化的、动态变化的。

技术上两者都用到检索，但侧重点不同：RAG 更关注切分、召回和引用；记忆更关注写入时机、冲突治理、时效和隐私。

实际系统里会组合：RAG 提供事实依据，记忆提供上下文和偏好。两者要分开存储和评估，不能混在一个向量库里，否则权限和生命周期会乱。

## 10. 怎么评估记忆系统的效果?

我会分四类指标。

第一，召回效果：该记住的信息，在需要时能不能取出来；用命中率、`Recall@K` 衡量。

第二，准确效果：取出来的记忆是否正确、是否过期、是否冲突；用准确率和人工抽检。

第三，任务效果：有记忆和没记忆相比，任务成功率、人工修改率、交互轮次有没有改善。

第四，污染与风险：错误记忆的比例、跨租户泄露、过期信息被使用的情况。

评估集要包含「应该记住」「不应该记住」「应该忘记」「应该更新」四类场景，而不只是正向召回测试。

记忆系统的价值最终体现在任务效果上，不能只看存储量或检索命中率。

## 11. 经验记忆/故障案例库怎么设计?

经验记忆不能只是保存聊天记录，要结构化。

我会存：问题现象、触发条件、诊断过程、根因、解决方案、验证结果、适用边界、来源和时间。最好再带一组特征标签，方便按症状检索。

写入时机是「问题确认解决之后」，而不是过程中。写入前要做脱敏和去重，避免把一次偶然事件当成通用经验。

检索时先按现象和标签召回，再按相似度和时效排序；复用时不能直接照搬，要先验证当前场景是否满足适用边界。

还要有反馈：这条经验有没有解决问题、有没有被修正。没有反馈的经验库会越来越不可信。

在排障 `Agent` 里，这类经验库的价值很大，但前提是「案例经过验证」，否则会把错误诊断传播得更快。

---

## 高频追问

- 记忆写入应该同步还是异步？失败了怎么办?
- 用户说「忘了这件事」，系统应该删到什么程度?
- 摘要和原始消息冲突时，以哪个为准?
- 长期记忆要不要让模型自己决定写什么?
- 记忆检索和 RAG 检索能不能共用一个向量库?
- 如何防止记忆被 Prompt Injection 污染?